

AGENT HARNESS · TECHNICAL TALK

Agent Harness Explained:

OpenCode, Claude Code, Codex & DeepSeek Harness

Context · Memory · Agent Loop · Multi-Agent · Tools · Self-Evolution

"Prompts are suggestions. Harness code is law."

— Mitchell Hashimoto · Co-founder of HashiCorp, creator of Vagrant & Terraform

A leading infrastructure engineer, writing on AI coding agents and Harnesses

Evolution: From Prompt to Context to Harness

Prompt Engineering

2022 — 2024

Making intent clear

Focus: single-output quality

Optimizing one LLM call

Zero-shot / Few-shot / CoT

Hard limit: single turn only

Context Engineering

2025

The right knowledge boundary

Focus: what the model sees

Managing the whole context space

Curation · Optimality · Evolution

Goal: minimal tokens × high signal

Harness Engineering

2026 —

Reliable, in control

Focus: long-horizon reliability

Deterministic infra around LLMs

Four layers · bounded behavior

Errors → fixable defects

How the three stack up

Stacking: Harness $\supseteq$ Context $\supseteq$ Prompt — each layer solves what the previous left open

Definition: deterministic infrastructure that makes probabilistic models reliable in production

Data: same model, swapping the Harness lifts Terminal-Bench-2 pass rate 52.8% → 66.5% · Next: Loop / Graph Eng.

Two Core Formulas

Production Agent = model potential – model entropy + Harness constraints

Optimal context = minimal tokens × highest signal-to-noise

Why the model is "non-deterministic"

Next-token prediction; output comes from sampling

Same input can yield different outputs

Sensitive to phrasing; can hallucinate

Sampling: temperature / top-p / seed

→ "entropy": behavior can drift

Why the Harness is "deterministic"

Built from deterministic code: state machines, rules

Same input → same output, repeatable

Testable, revertible, observable

Permissions, retries, timeouts are hard rules

→ "entropy reduction": bounding uncertainty

Sources Context formula — Anthropic, "Effective Context Engineering for AI Agents", Sep 2025

Entropy framework — distilled from industry Harness Engineering practice

The model generates; the Harness constrains — every token must earn its place.

CHAPTER 01

Chapter 1 · Generic Harness Architecture

Four-layer architecture

Shared abstraction of six projects: Claude Code / OpenCode / Codex / Hermes / DSH / Pi

Six-project comparison

One table for commonalities and differences

How to read this deck

Generic mechanisms → six-project contrasts → engineering details

Generic Harness: Four Layers Shared by Six Projects

L1 推理与编排层 —— 决定怎么循环

重点: QueryEngine · queryLoop · TAOR · multi-Agent

QueryEngine 会话层: submitMessage 接收消息 → 构建 SystemPrompt → 处理用户输入 → recordTranscript 持久化

queryLoop 执行层: LLM 流式调用 → 状态机 → 工具执行 | multi-Agent: Subagents / Swarm / Coordinator

L2 上下文与记忆层 —— 决定看什么信息

重点: Context 上下文管理 · Memory 系统 (四分类 · AutoDream · CLAUDE.md / MEMORY.md)

Context: 静态段 + 动态段缓存 · 四阶段流水线压缩

Memory: CLAUDE.md 记忆文件 · MEMORY.md 索引 · AutoDream 闲置时后台总结

L3 工具与安全执行层 —— 决定能做什么

重点: Tool 工具系统 43+ · command 命令系统 100+ · 权限引擎 · 系统级沙箱

Tool: BashTool · File Read/Write · Agent Tool · MCP · Skill · Grep | command: /help /init /review /plan /diff /cost

安全控制: 权限引擎 · 系统级沙箱 · Bash AST 分析 · 自动意图分类器

L4 支撑与基础层 —— 决定跑得稳吗

重点: 多端交互入口 · AgentSDK · Session · Provider · 可观测性

交互层: CLI · TUI · Desktop · Web · IDE 插件 · AgentSDK (异步生成器事件流 · 同进程异步调用)

Session 会话 / Provider 模型接入 / Config 配置 / 可观测性

Claude Code real architecture mapped to the four-layer abstraction

Six Projects: Commonalities and Differences

Same model, different Harnesses, different results · Memory, permissions, multi-agent and self-evolution differ most

Dimension	Claude Code	OpenCode	DSH	Codex	Pi	Hermes
Language	TS/Bun	TS/Bun	TS/Bun	Rust	TS	Python
Arch.	Modular services	Event-driven	Plugin microkernel	Thread/Turn	Minimal loop	Self-evolving loop
Tools	43+	16-18	40+	config	4	40+
Compaction	window-13K	count≥usable	dual trigger	90% dual scope	-16384	×50%
Memory	memdir+index	none	none	SQLite+git	none	MEMORY+FTS5
Perms.	6-level+AST+classifier	3-layer Ruleset	Guard+ToolGuard	Approval+Sandbox	none	write-approval 3-state
Self-evo.	autoDream	none	none (composable)	two-phase pipeline	none	curator+RL
Multi-agt	Task+SendMessage	Task subagent	6 provider	Thread fork	none	no recursive subagents

One-line intros in speaker notes · differences cluster in memory / multi-agent / self-evolution

Shared: four layers · loop · compaction · fail-closed permissions — differences chapter by chapter

Chapter 2 · Context System

Three constraints

Lost in the Middle · Context Rot · $O(n^2)$

Compaction

Trigger → threshold → rebuild, end to end

Slimming in practice

Fixed per-turn context typically halves (mechanism later)

Why More Information Makes Agents Worse

Three experiment-backed constraints form the theory base of Context Engineering

① Lost in the Middle

位置衰减 · 效果问题：漏答

开头 75% · 结尾 75%

中间 35% ↓

机制：关键信息居中被忽视
(Stanford 2023 · U 型曲线)

对策：关键信息放首尾
长文档引用锚定定位

影响：漏答 · 中段信息盲区

② Context Rot

上下文腐化 · 效果问题：答得差

1k → 4k → 16k → 64k

质量 ↓↓

机制：无关内容稀释有效信号
(Chroma 2025 · 与长度相关)

对策：压缩 · 折叠
只留相关内容

影响：效果——答错 · 漏答

③ Attention Budget

注意力预算 · 成本问题：算得贵

n 个 token 两两计算

注意力 → $O(n^2)$

机制：两两注意力，计算量平方增长
预算被上下文长度吃掉

对策：控规模 · 分层缓存
KV 缓存复用

影响：成本——延迟 · 费用

LIM · Rot = 效果衰减 (漏答 / 答错) · Budget = 成本激增 (延迟 / 费用) ——上下文是稀缺资源，压缩是必需

Attention is scarce — adding irrelevant content actively weakens the signal of what matters

Context & Prefix Caching — OpenCode measured

一次请求上下文构成 (OpenCode 实测 · 无用户消息 · 合计 15,898 tok = 63,229 字符) · 按静态 / 动态分区：



Tools 13 个 = 上游 9 (bash · read · edit · search · write · webfetch · todowrite · question · skill) + Bocom 新增 4
Bocom 新增：memory_search (记忆检索) · project (跨项目) · task (子代理) · schedule (定时任务)

占比：System 30.2% vs Tools 68.9%——工具说明是每轮固定成本大头 (►Bocom Lite 第一刀) · token 估算：中文×0.75+非中文×0.2416，校准至 15,900

Leftmost-prefix rule (Prompt Caching)

Cache hits require byte-identical prefixes · hits are free · any change (even whitespace) invalidates · cache_control marks the boundary

Practice: mutable content goes last · CC forked subagents inherit the parent tool array · index folding works the same

68.9% of tokens are tool descriptions — the recurring cost driver; a stable prefix = cache hits = cheaper and faster

Context Compaction: How Projects Do It

Compaction = summarize old history when the window fills, keep recent turns verbatim — differences: when / how / what

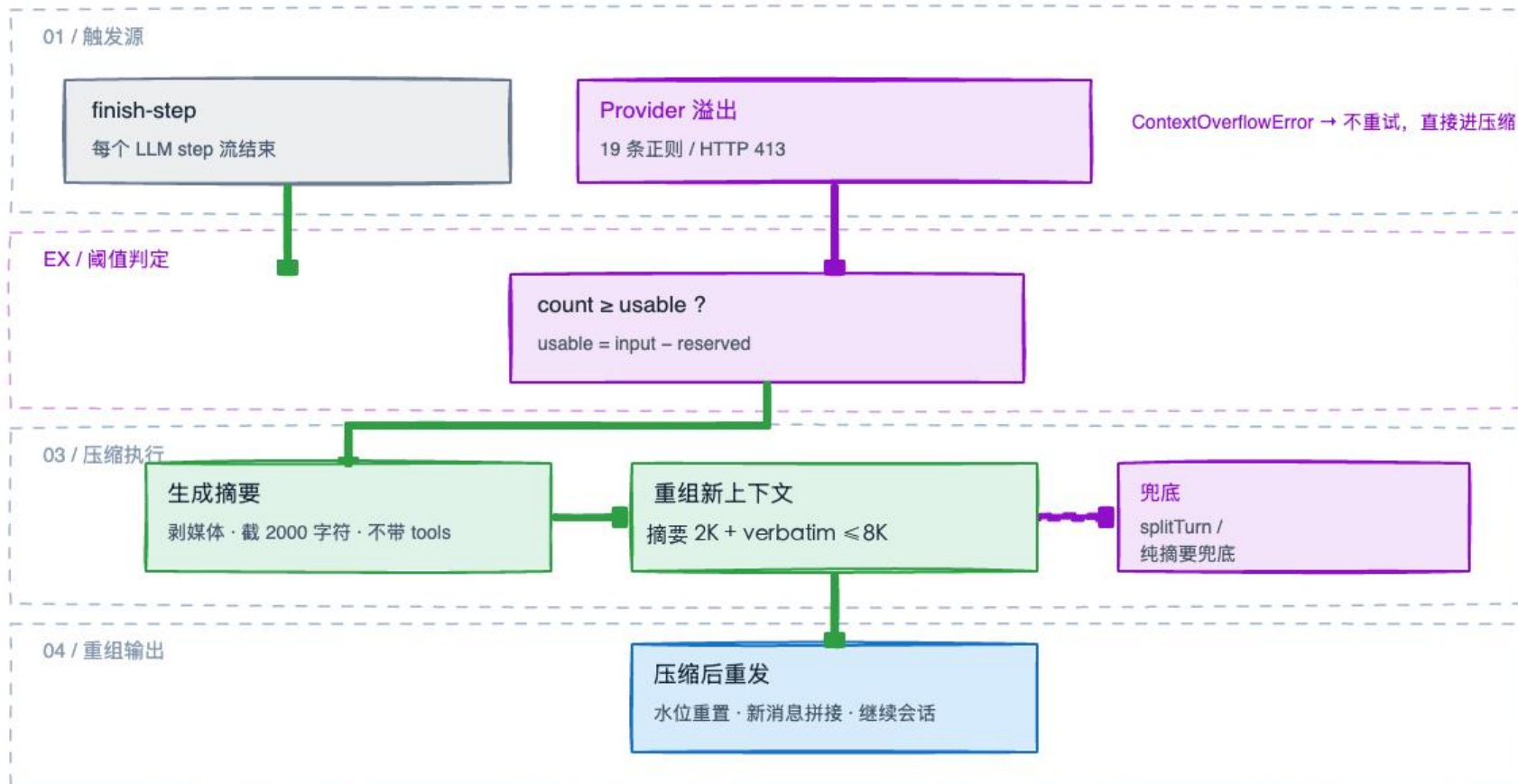
Project	When (trigger)	How (free first, then paid)	What to keep · signature
Claude Code 5-stage progressive (10K left)	Acts when window nearly full	Free slimming first: cap tool results → prune old history → clear tool outputs Only then a subagent writes a 9-part summary	Summarize past details in main session cache — cheap (not reusing wastes 98% of cache)
DSH Two-tier + spill	Belt and braces: check every step plus overflow-error fallback	Clip big tool outputs to head/tail snippets then LLM summarize if needed	Huge outputs (>50KB) go to external files model sees preview + locator only
Codex LLM summary	Acts at 90% of window (90% threshold)	Direct LLM summary Two accounting modes: full / delta only	Count only the delta when the prefix is stable avoids repeated compaction
Pi Cut-point algorithm	Acts at 16,384 left (keeps 20,000 verbatim)	Cuts only at message boundaries never mid-tool-call	guarantees replay integrity single file ≈850 lines
Hermes Direct summary	Halves history on overflow (×50%)	Direct LLM summary (simplest) no tiers, no cut points	Persists memory before compacting (with autoDream)

Shared: recent turns stay verbatim · free slimming first · pre-check + overflow fallback

OpenCode implements both pre-checks and overflow fallback with a usable budget line — see next page

OpenCode Compaction: usable Budget + Dual Triggers

Upstream implementation: usable budget line · 8-part summary template · recent-verbatim window · compact on overflow, no retry



Key numbers

usable = input limit - reserved (e.g. 118K-36K=82K) · verbatim = min(usable×25%, 8K) · new context = 2K summary + verbatim

Compared: CC window-13K · Codex 90% dual scope · Pi -16384 · Hermes ×50% | OpenCode = usable budget line (upstream)

Context Slimming: Three Generic Cuts

Why slim the fixed cost

System prompt + tool descriptions are resent every turn · anything irrelevant is pure waste
Impacts: token cost · attention budget (see page 8)

Principle: tiered trimming, never one-size-fits-all

Scenario decides the tool surface: office / coding / review
Test: useful here? If not, switch off, add an equivalent

Cut 1 · Trim tool descriptions per scenario

Example: bash description is 7.3KB; office mode strips git-commit / PR sections
Trim below half; examples 6 → 2
Keep: invocation rules · truncation behavior · parallelism hints
Rephrase stale references · merge duplicate lines

Cut 2 · Progressive disclosure in the prompt

Send each list once (drop verbose duplicates)
Fold memory index → one line; fetch body on demand
Compress examples and quote do/don't cases
Files stay intact · read via tools when needed

Both cuts target what is resent every turn — only the injection changes; files and capabilities stay intact

Tool descriptions are usually over half (68.9% on page 9) — the highest-yield, lowest-risk cut

Benefits

Fixed per-turn tokens typically drop 30–50% (scenario-dependent)
Savings scale linearly with turn count
Sharper instructions → fewer tool-selection mistakes
Cache-friendly cuts only touch tail segments

Limits and costs (be explicit)

Over-trimming removes needed context — err on the side of keeping
Every removed capability needs an equivalent (script / subagent)
Switchable modes: coding restores the full tool surface
Validate defaults with before/after measurement

Slimming ≠ removing: tier by scenario, provide equivalents, be explicit about trade-offs

Chapter 3 · Memory System

Lifecycle

Short-term → mid-term → long-term flow

Freshness & forgetting

Conflict resolution · scoring gates · active pruning

External brain

Knowledge base vs memory · two implementation routes

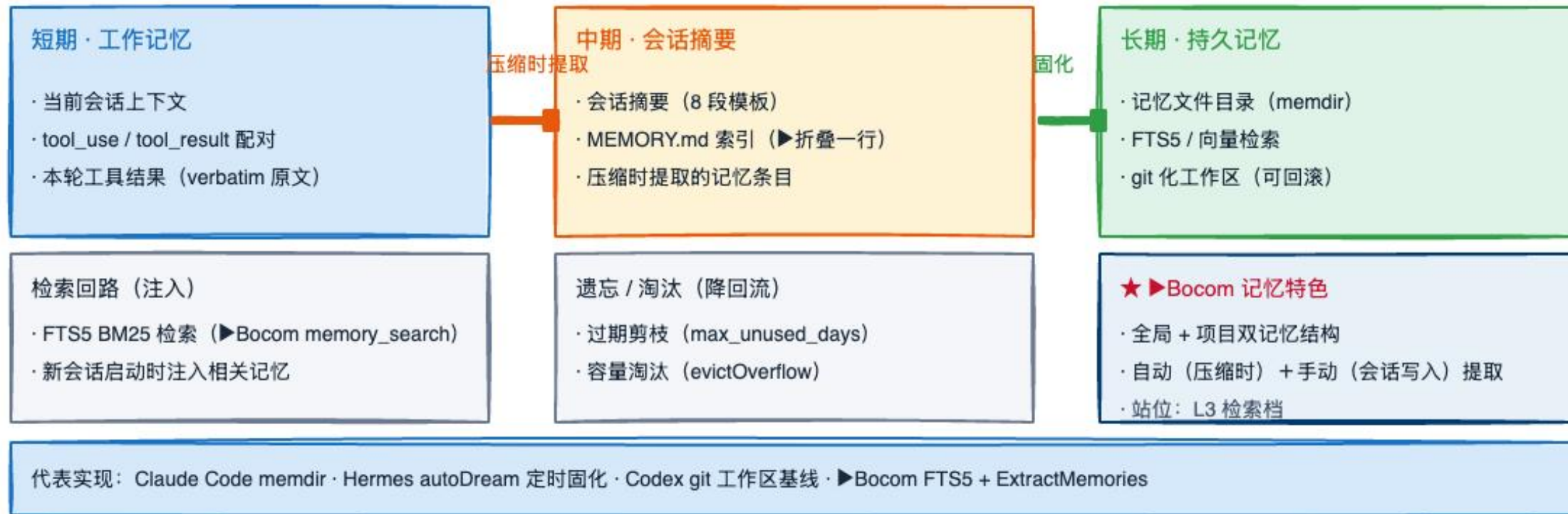
Three injection modes

Full injection / resident index / retrieval · six-project contrast

Memory Lifecycle: Short → Mid → Long Term

Working memory is distilled into summaries and persisted; retrieval re-injects on demand; forgetting = pruning + eviction

Agent 记忆生命周期流转



▶ Bocom = L3 检索档: 索引折叠 (省 token) · FTS5 BM25 检索 (保能力) · 压缩时自动提取 (省调用)

Contrasts

Claude Code memdir · Hermes autoDream scheduled persistence · Codex git-workspace baseline

Memory Freshness & Forgetting: Who Owns Quality

Stored memories go stale, conflict and duplicate — three source-level quality-control mechanisms

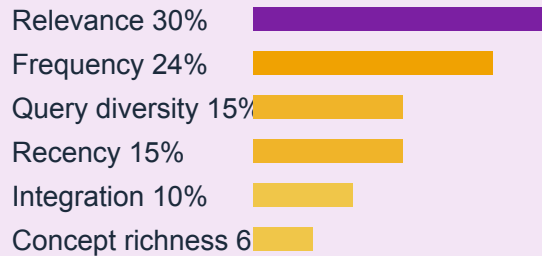
Claude Code · Auto Dream

- ① Trigger: user idle $\geq 24h$
(10-min scan throttle)
- ② LLM arbitration: resolve conflicts, dedupe
- ③ Newest entry wins; old fades out
- ④ Hard index caps: ≤ 200 lines · ≤ 150 chars each
Limit: exact keyword match — paraphrases miss

Freshness: background, while idle

Flow: preflight → time gate → scan

OpenClaw · 6-D weighted scoring



Gates: $\text{score} < 0.8$ · $\text{recalls} < 3$

Graduated: Light Sleep logs signals first
→ Deep Sleep filters before writing MEMORY.md
Re-reads logs before writes

Generic rules (from the three)

- ① LLM arbitration, never blind rewrite
Resolve conflicts, dedupe; newest wins
- ② Forgetting: mark → count → evict
Expired → strikes, then evict
- ③ Write gates: score / recalls / sources
Below the gate, drop — quality over quantity
- ④ Re-read source data before writing
Disk/logs are the truth, not the memory

Common: quality control is a separate stage

Codex · Pruning in a two-phase pipeline

Phase 2: prune when last_usage exceeds max_unused_days
High usage_count kept first · git-diff baseline for rollback

Hermes · Periodic freshness

Periodic Nudge: a "learn now" signal every 10 turns
Background subagent consolidates · MEMORY.md/USER.md frozen snapshots

Freshness = LLM arbitration + gates; forgetting = active eviction — no QC means dumber memory

The External Brain: What, Where, How

External brain = knowledge outside the model: index, retrieve on demand, inject — for what the model never saw or cannot fit

Where it sits and what it does

Retrieve Top-K from the knowledge base

→

Context assembly (system + memory + knowledge)

→

LLM reasoning

Value: ① beyond the window — what does not fit becomes queryable ② private knowledge — enterprise docs / code ③ less hallucination — answers cite sources

Memory (first, already built)

Item-level · session-distilled · tens of entries

Keyword search suffices (SQLite FTS5 + BM25)

Knowledge base (external brain)

Document/code-level · large · pre-existing assets

Needs semantic search / graphs / source citations

How it works (four steps)

① Ingestion

Chunk docs / code / PDFs

→

② Indexing

Embeddings / BM25 / graphs

→

③ Retrieve

Top-K by relevance

→

④ Inject

Into context · with citations

Route A · Cloud / RAG

Examples: Mem0 · Letta (ex-MemGPT) · vector DBs

How: text → embeddings → similarity Top-K

Pros: strong semantics · large unstructured corpora

Costs: vector-DB ops · embedding models · network

Route B · Local lightweight

graphify (120k★): code/docs → local knowledge graph

tree-sitter AST parsing · no vector DB · fully local

claude-obsidian (15.1k★): Obsidian as the knowledge base

BM25 search · source citations · plain Markdown

Guidance: item-level memory is Memory with local keyword search (FTS5 + BM25); add semantic RAG (next page) for large unstructured corpora

Three Memory Injection Modes

How stored memory enters the context — resident cost vs recall, six projects across three modes

① Full injection

Rule files live in context in full
CLAUDE.md / AGENTS.md sent every turn

Pro: always visible, never missed
Cost: grows linearly with content
Crowds out the attention budget

Seen in: most rule files · Pi full text

② Index + on-demand read

Inject only the index (tens of tokens/entry)
Bodies stay on disk, read whole when needed

Pro: resident cost an order smaller
Cost: one extra hop; index quality decides
hits — descriptions must say "when to use"

Seen in: Claude Code memdir + MEMORY.md index

③ Retrieval injection

FTS5/BM25 (or vectors) fetch Top-K
Only matching entries enter the context

Pro: zero resident cost, scales to 10k+
Cost: paraphrases can miss recall
Keyword search ≠ semantic search

Seen in: FTS5 ecosystems · RAG KBs

Six-project positions (injection)

- Claude Code: memdir + resident index + Sonnet picks 5 (mode ②, semi-automatic)
- Hermes: pre-compaction flush + autoDream persistence (②+④, automated); Codex: two-phase pipeline + git baseline
- OpenCode: full-injection rule files (①), memory via plugins; Pi: minimal full text (①)

Order: start ① small → ② as content grows → ③ at scale — keep the recall net at each step

Chapter 4 · Agent Loop

Loop thinking

ReAct → TAOR · three engineering styles

Main loop

Six-step pipeline: assemble → infer → parse → guard → execute → feed back

Optimization patterns

Snapshot sampling · no-change short-circuit · write opt-in · slow-environment fallback

Principle

Only write tools need a baseline hash

Agent Loop: from ReAct to TAOR, three styles

Two names, one idea; the pipeline is on the next page — here: the mapping and three styles

TAOR stages → generic implementation

Think → streaming LLM call (after assembly)

Act → parallel tool execution (≤ 10 overlapping)

Observe → feed results back (tool_result)

Reflect → summarize / compact (next assembly)

OpenCode family: serial main loop · parallel tools

Serial by choice: replayable, debuggable, deterministic state

One step = one LLM call + its tool runs

Three engineering styles (from source)

Claude Code: six-step pipeline

autocompact forks a subagent for the 9-part summary

DSH: ReactLoopAgent itself is a plugin

Hot-swappable loop · event-sourced fork / replay

Pi: flat double loop · 25 hook points

≈850-line single file · minimal yet extensible

OpenCode: event-driven · single-threaded serial

One full conversational turn

Message → assemble context → LLM thinks, picks tools → parallel run → results back → think → answer → persist

Every turn counts tokens, latency and tool calls — the raw data for the observability chapter (measure first)

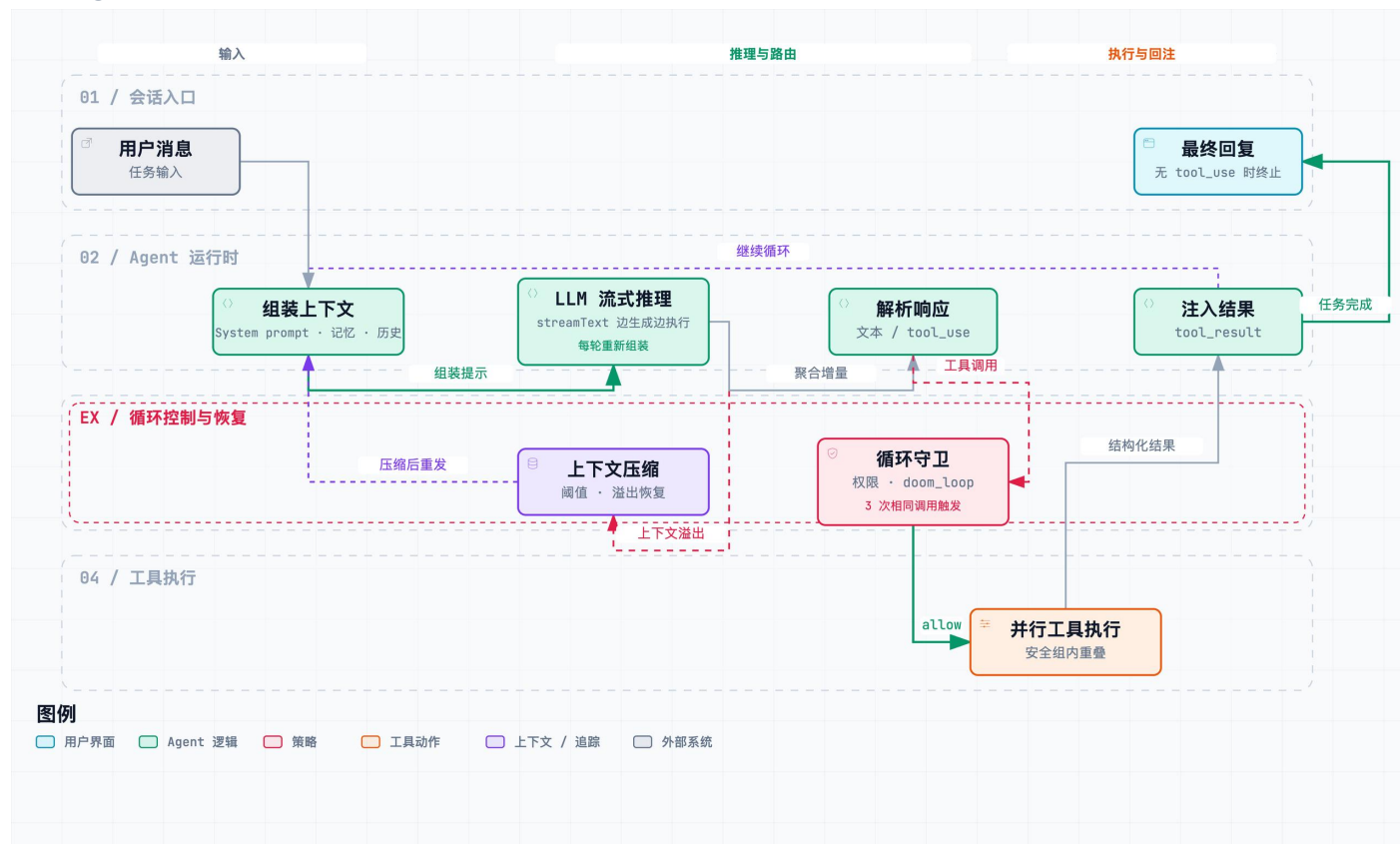
Overflow bypass: any step over budget goes straight to compaction (see the compaction chapter)

Prompts are suggestions, Harness code is law —

every step of the loop is deterministic code: permissions, execution, assembly, compaction. The LLM only Thinks

Agent Loop: The Generic Main Loop

Assemble → stream → parse → guard → execute in parallel → feed back; on overflow, compact and resend



Contrasts

Claude Code six-step pipeline · DSH six-state TurnEndReason (sticky max-tokens) · Pi flat double loop, 25 hooks

Agent Loop: Snapshot & Observability Optimizations

Snapshots = git track/patch for session rollback · events = staged activity streams · slow environments = timeout instead of hang

① Snapshot optimization · three cuts

Problem

Snapshot follows every tool → hundreds of git calls per turn
Read-only tools trigger full snapshots too

Three optimizations

- ① per-message boundary
Snapshot once per message, not per tool
- ② lastHash cache
diff-files check → skip when unchanged
- ③ Writable Opt-In
Only write tools snapshot

git calls ↓1–2 orders of magnitude

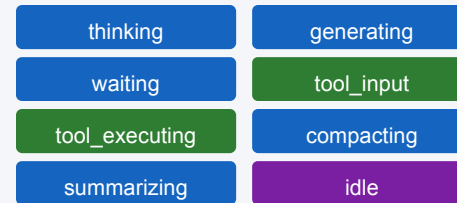
Read-only sessions: end-of-turn cost zero
Core: only writes need a baseline

fail-open trade-off: bash is not a write tool by default
rare bash-written files survive rollback
accepted

Independent gains, best combined

② Staged event stream · 8 stages

8 stages



29 activity event types

Published asynchronously

Event bus + async fiber per subscriber

negligible main-loop overhead

TUI rendering

- Breathing light: current stage
- Waterfall: per-step timings
- Subagent panel: up to 3 [BG]

Key

- Bus + forkIn make observation free
- async publishing never blocks the loop

③ Slow environments · timeout fallback

Three symptoms

git hangs on NFS · scans stall · snapshot writes crawl
Blocked syscalls defeat app-level catch

Three tools

- ① Configurable timeouts
AbortController around every child process
- ② Parallelize
parallel diff/ls-files · Grep on a fiber
- ③ Diagnostic probes
13 probes · 50ms threshold

Configured timeouts

git 30s · snapshot 60s
grep 180s · hook 30s

Core

Turn "wait forever" into
"degrade gracefully on timeout"

Key

- make slow environments observable and tunable
- never let one slow call sink the session

One system: snapshots (write-triggered) + event stream (async) + timeout fallback (slow environments)

Measurement comes first: without a structured event stream there is no before/after evidence

Observability & Event Sourcing

Measurement precedes optimization — turn execution into structured events to locate issues and prove before/after

Event stream

Split a session into staged events:

thinking / generating / waiting /

tool_input / tool_executing /

compacting / summarizing / idle

each activity carries duration and token counts

async: never blocks the loop

TUI/IDEs render progress and timings from it

Uses: find bottlenecks · attribute cost

Event sourcing

Deeper: every input the model saw

has a log counterpart (visible $\iff$ logged)

a session = a replayable event sequence

Three capabilities follow:

- fork: branch a new session from any event
- replay: deterministic replay debugging
- time-travel: inspect any prior step

Seen in: DeepSeek Harness

Telemetry export (OpenTelemetry)

Bridge internal events to standard telemetry:

- traces: request-level tracing
- metrics: tokens / latency / success rate

Claude Code officially supports OTel export;
structured events come before telemetry

Enterprise value: dashboards · alerting ·
cross-team cost attribution and audit

Key: keep the event schema stable

Three levels: event stream (visible) → event sourcing (replayable) → telemetry (managed)

Chapter 5 · Multi-Agent

Why multi-agent

Three bottlenecks · isolation is the point

Four orchestration patterns

Workflow / Supervisor / Hierarchical / Swarm

Communication

Request-reply → peer-to-peer → mailbox → A2A

Long-lived subagents + persistent mailbox

Long-lived subagents · async decoupled send/receive

Why Multi-Agent: Three Bottlenecks

One agent doing everything is one person coding, researching and reviewing at once — nothing done well

Bottleneck 1 · Context bloat

A full notebook hides what matters

All tool results share one window:

- ten search rounds, all results retained
- old data fills the window → new asks diluted
- compaction fires → early details lost

Fix: subagents take the dirty work

- Searches go out; conclusions back
- Evidence: CC microcompact

Analogy: you clear drafts off your desk

Bottleneck 2 · Tool choice

Too many keys, wrong doors

Beyond 20 tools, selection errors rise:

- each tool is one more option on the quiz
- interference → wrong tool → wasted tokens

Fix: subagents carry only a relevant subset

- Trim to the scenario minimum (Pi keeps 4)
- Pi: just 4 (the minimal counterexample)

Analogy: nobody carries 20 keys on one ring

Bottleneck 3 · Conflicting roles

Player and referee in one

Divergent exploring vs rigorous reviewing

- in one system prompt they fight each other
- coding wants freedom; review wants rigor

Fix: reviewer subagent with maxTurns=1

- shortest life · independence only
- refusing a call = task failed (Sonnet 4.6)

Analogy: writer ≠ reviewer

Isolation is the essence of multi-agent

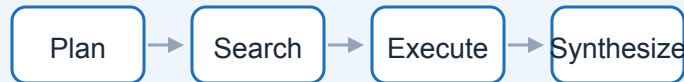
Each subagent gets its own context, tools and permissions — not more brains, more clean desks

Gains = context isolation + failure isolation + parallelism — the pattern lands on the Actor+Inbox page

Multi-Agent: Four Orchestration Patterns

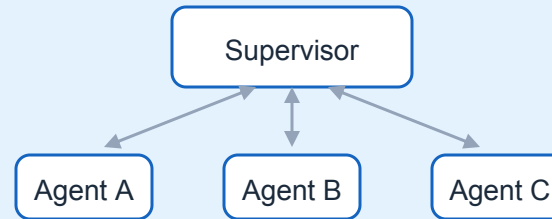
Structural differences of four patterns — direction and centrality are the story; core elements on the right

Workflow · fixed pipeline



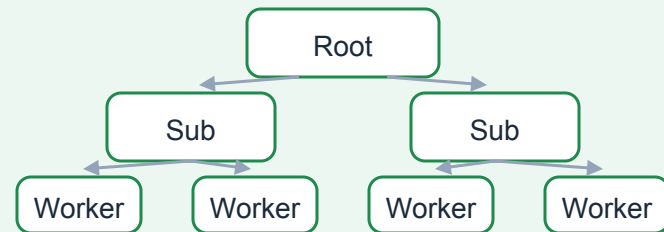
Fixed order · single path · predictable

Supervisor · central dispatch



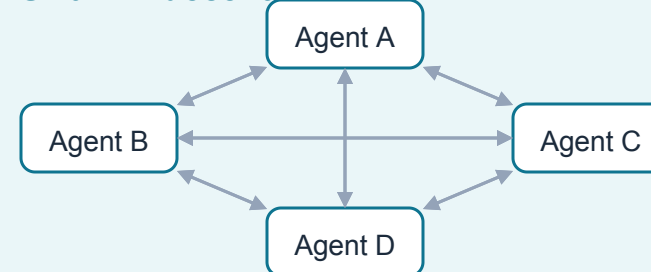
Dynamic routing · central dispatch · synthesis

Hierarchical · tree



Layered decomposition · failure isolation · scale

Swarm · decentralized



Peer-to-peer · self-handoff · high autonomy

Core orchestration elements

① Task decomposition

Workflow pre-coded · others decompose live

② Three subagent lifetimes

Detached — own window, starts from zero

Fork — copies parent context

Persistent — long-lived session, append anytime

③ Two result-return styles

Report — conclusions only

Inbox async — fire-and-forget, decoupled

Cost & fault tolerance

Cost: subagent = context, tokens multiply

Faults: redispach · isolation · redundancy

Long-lived + mailbox = controllable Swarm

Choosing Predictable → Workflow | dynamic routing → Supervisor | scale → Hierarchical | autonomy → Swarm

In production CC Task+SendMessage / DSH 6 providers / Codex Thread Manager fork / Pi none | long-lived subagents + mailbox = controllable Swarm

Note OpenAI Swarm: no lead agent — **the current agent handoffs**; CC Agent Teams: a **Team Lead dispatches** (Supervisor variant). Line = **central delegation**

Agent Communication: from Request-Reply to A2A

Four modes, rising complexity: communication capability bounds orchestration freedom — from blocking calls to standard protocols

① Parent-child request-reply

Parent starts → blocks → result returns via tool_result
Parent stalls; simplest semantics
No comms infrastructure; result is just LLM output
Fits: one-level split, result needed immediately
Seen in: CC Task / DSH fork (one-shot)
Limits: nesting multiplies tokens; long tasks block the parent until every child returns
child crash → parent errors; no retry
Risk: parent waits too long → timeout → turn fails

② Peer-to-peer messages

Agents message each other directly; parent unblocked
Transport only; orchestration on the model
Seen in: CC SendMessage; Swarm relays
No persistence: in-memory only, lost on restart
Seen in: CC SendMessage / Swarm
Limits: no acks; ordering not guaranteed
No retries; the model handles timeouts and resends
Swarm: relay-style autonomy, no broker
Risk: silent message loss; hard to debug

③ Mailbox / broker

Persistent mailbox; fire-and-forget
No blocking either way; results arrive async in the Inbox
A dedicated DB table brokers messages
Architecture: Actor registry handles wait → wake
Long-lived + persistent broker → controllable Swarm
Pros: survives restarts; traceable across turns
Dispatch → keep working → read later
Fits: splittable tasks whose results can wait

④ A2A protocol (industry standard)

Agent2Agent, released by Google in 2025
Three cores: Agent Card (self-described skills)
+ task lifecycle management
+ message / artifact exchange
Transport: JSON-RPC over HTTP / SSE
Key value: cross-vendor interop (no lock-in)
Linux Foundation (Jun 2025) · complements MCP
Trend: in-process IPC → standardized network APIs

Side-by-side comparison

Capability rises left to right; so does complexity

Dimension	① Sync	② P2P	③ Inbox	④ A2A
Latency model	blocking	async	async durable	network async
Persistence	✗	✗	✓ DB	✓ optional
Result path	tool	stream	poll	state machine
Retry	✗	✗	✓ framework	✓ protocol
Interop	in-process	same framework	same framework	✓ cross-vendor
Complexity	lowest	low	mid	high
Seen in	CC Task	SendMsg	Inbox-style	A2A
Evolution path	Actor	—	Inbox	watch

Key insights

①+② are in-process; cross-machine needs ③ or ④
③ is the stepping stone to ④; Inbox upgrades smoothly to A2A
Order: start with ① → hit limits → ③ → standardize on A2A

Trend: in-process IPC → standard protocols — the mailbox route leaves room for A2A

Landing pattern: long-lived Actor + persistent Inbox

Actor model (async messages + state): Actor registry + persistent Inbox table · cf. CC Task, DSH startContinuable

Actor registry

Subagents register as Actors; parent continues
Registry owns wait/wake: block only on need
Contrast: CC Task blocks once for a report

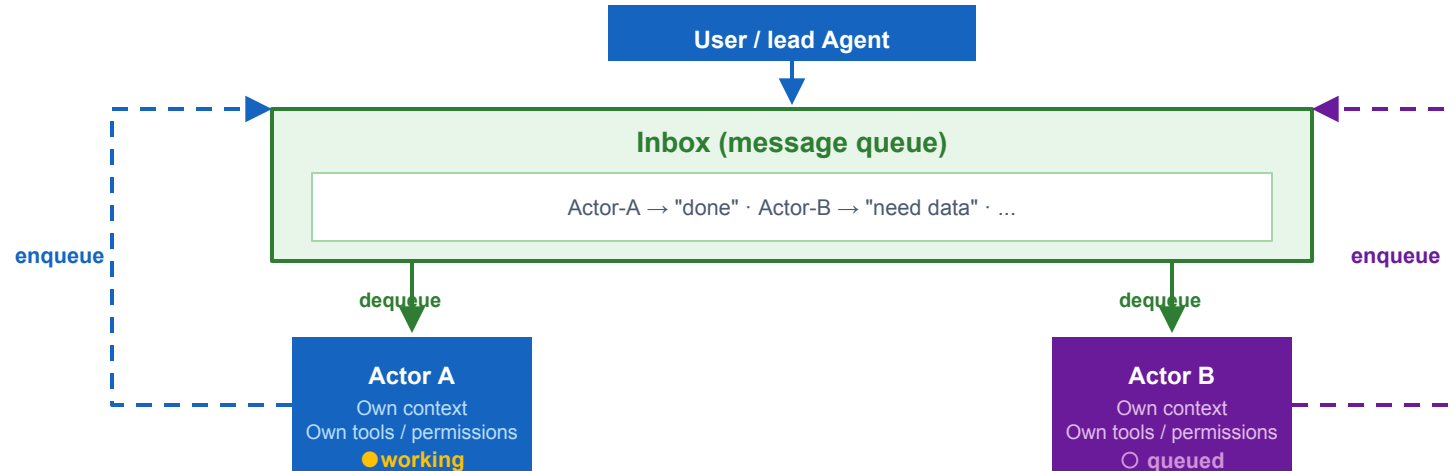
Inbox

A dedicated Inbox table brokers messages
Send and continue; results land async
A controllable Swarm: P2P exists but goes through the broker

Compared to two projects

CC: Task blocks; SendMessage adds P2P
DSH: fork (one-shot) / startContinuable (persistent)
Persistent + brokered: long-lived Actors with Inbox decoupling

Actor + Inbox workflow (closed loop)



Parallel without chaos: messages are brokered durably; the registry owns waiting and waking

Typical use: Send broad code searches or option research to subagents while the loop continues; read results from the Inbox when needed
Rule of thumb: async pays only when the task splits and results are not needed immediately

Chapter 6 · Tool Integration System

Registry · parallelism · permissions

fail-closed: default to ask · loop guards as backstop

Skill

Three-level progressive disclosure

MCP

The standard plug-in protocol · vs Skill

Tool design practice

Description writing · param schemas · model-facing errors

Tool System: Registry · Parallel Execution · Permissions

OpenCode: 16 built-in tools + plugin extensions · Tool.define() registry · AI SDK streamText concurrency · 3-layer Ruleset permissions

① Three-layer registry + plugin override

16 built-in + {tool,tools}/* files + plugin.list()

Plugins shadow built-ins by ID (plugin-shadows-built-in)

Writable overrides inherit the revert snapshot baseline

② Dynamic trim per model/agent

GPT → apply_patch · vision-capable models → show multimodal

Slim mode: glob+grep merged into search

activeTools feeds streamText — the model-visible set

③ Streaming concurrency: one streamText response can emit many tool-calls → AI SDK invokes them in parallel

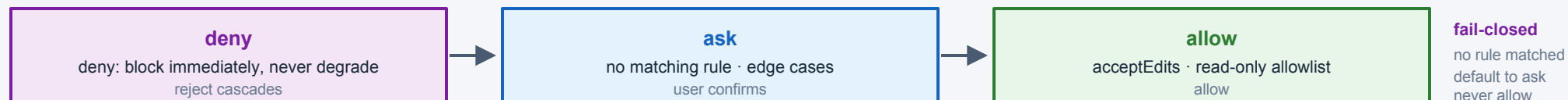
- processor.runningTools tracks in-flight tools · the model decides: independent calls run parallel, dependent calls serialize
- activeTools comes from ToolRegistry.tools(model) — only tools the current model/agent should see

④ Loop guard (common practice): stop repeated identical calls

Three identical tool+params calls in a row → permission prompt; a deny aborts — no burning tokens in loops

⑤ Permissions: deny → ask → allow (permission/next.ts, 518 lines)

- static rules → acceptEdits → read-only allowlist → ML classifier, evaluated layer by layer (evaluate is 15 lines)



Trim registry per scenario · the model decides concurrency · loop guard + fail-closed permissions — three safety nets

Tool classes: files read/write/edit · shell bash · search glob/grep · web fetch · tasks task/todo/question · flows skill/plan · plugins extend further

Tool Design Best Practices: description & schema

Tools are APIs — the model decides whether and how to call from the description and schema; the writing decides call quality

description: four elements

- ① What: one line for the functional boundary
- ② When: trigger scenarios + typical examples
- ③ When not: division of labor with alternatives (avoids mis-selection)
- ④ Behavior: output shape · truncation rules · side effects

Bad: just "run a command" — the model cannot tell when to pick it

Good: "use read to open files; this tool only locates"

Parameter schema principles

- ① Fewer params: every parameter is one more quiz question
- ② Consistent names across tools (path / pattern / mode)
- ③ Enums over free strings (mode: files|content)
- ④ Minimal required: default optional params and document them

Poor params → model guesses → call fails → retries burn tokens

Merging tools (glob+grep→search) removes selection misses

Errors are model-facing

Tool errors are feedback for the model — make them actionable:

"file not found" → attach nearest candidate paths

"param invalid" → state the valid range

Errors enter context; the model self-corrects next turn

Human-only stack traces are noise to the model

Cost & determinism

- ① Truncation in the contract: spill to disk + return a locator
- ② Idempotency: is a repeat call safe (annotate write tools)
- ③ Timeout/cancel semantics: the model can expect interruptions

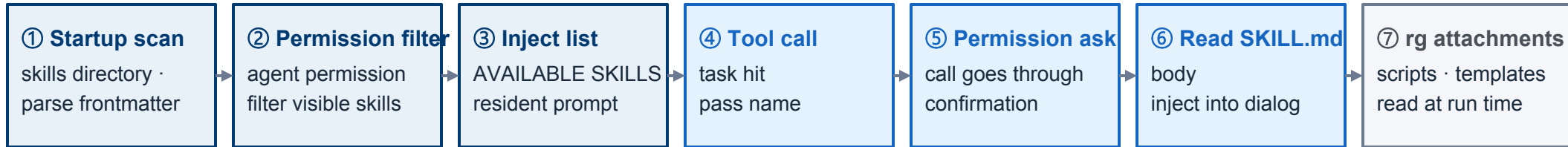
The description is a fixed cost resent every turn —

padding it taxes every single call forever

Good tools = clear description + minimal params + actionable errors

Skill Progressive Loading: Three Levels · OpenCode Flow

A Skill = a knowledge pack · only used Skills fully load (Anthropic spec) · flow = OpenCode



颜色 = 披露层级: ■ Level 1 · metadata resident ■ Level 2 · body on demand ■ Level 3 · attachments at run time

Level 1 · metadata resident

name + description enter context (tens of tokens)
the model self-selects from the description
discovery: scan ~/.claude/skills/ and similar paths
resident cost = skill count × tens of tokens
Writing: description = "what + when" + trigger words

Level 2 · body on demand

On hit, read SKILL.md: frontmatter + Markdown
required name · description; body ≤500 lines
Structure: What I do / When to use / Steps
DSH variant: three-stage progressive disclosure
Writing: push details to attachments · script fixed ops · one :

Level 3 · attachments

templates / scripts / reference files
referenced by relative path, read at run time
One skill, one job — avoid kitchen sinks
Hermes: auto-generates skills on criteria
Permission: external_directory widens access

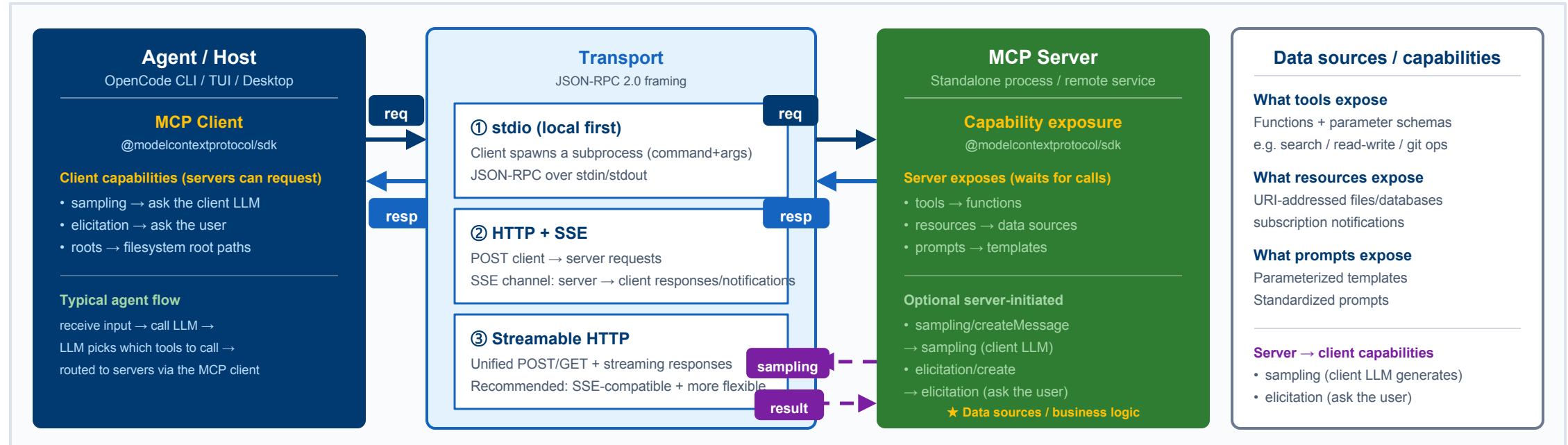
Same family: Send each list once (drop verbose duplicates) · fold the memory index the same way — resident index only, body on demand

The flow above is OpenCode native: scan → filter → inject list → skill tool → permission ask → read body → rg attachments

Completeness × token economy: only the index is resident (tens of tokens); unused skills cost zero

MCP: Bidirectional JSON-RPC · stdio / HTTP+SSE / Streamable HTTP

Open-sourced by Anthropic, Nov 2024 · mcp/index.ts ~1,000 lines · Server exposes tools/resources/prompts · Client offers sampling/elicitation/roots



What does an MCP Server expose?

tools · callable functions for the agent

Analogy: API endpoints. Each tool has a name, description and schema.
e.g. `github.create_issue(repo, title, body)`

resources · read-only data sources

Analogy: filesystem / database views, addressed by URI.
e.g. `file:///repo/README.md` · subscribe to changes

prompts · parameterized prompt templates

Analogy: mail-merge templates provided by the server.
e.g. `code_review.pr(diff, focus_area)`

Handshake lifecycle

- ① Client → Server: initialize (version + client capabilities)
- ② Server → Client: server capabilities
- ③ Client → Server: notifications/initialized
- ④ normal operation (both sides may send requests/notifications)

JSON-RPC 2.0 messages

request: `{ "jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": { "name": "...", "arguments": {...} } }`
notification: `{ "jsonrpc": "2.0", "method": "notifications/...", "params": {...} } (no id, no response)`

Chapter 7 · Agent Self-Evolution

Two RSI paths

Rewrite weights vs rewrite Harness code

Three-level path

Context → Workflow → the Harness itself

Memory ladder

L1 handwritten → L5 self-managed · six projects

Two more routes

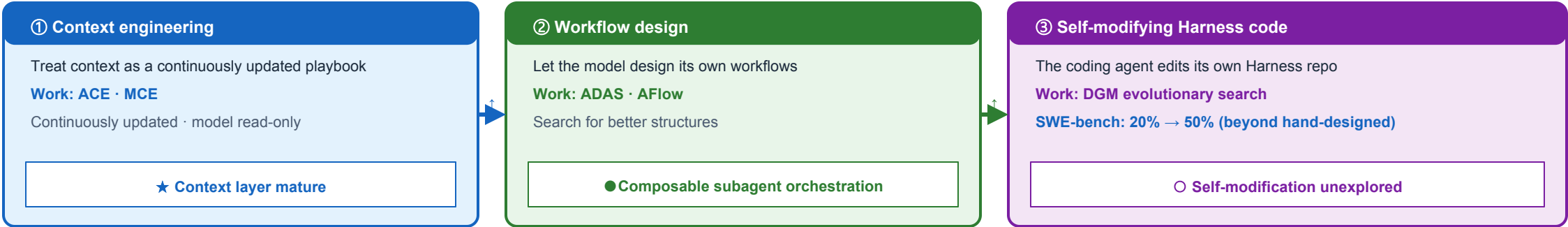
DSH restructurist · RL parametrization

Industry View: Self-Evolution · RSI and the Harness Path

RSI (Recursive Self-Improvement, 递归自我改进) + 翁荔 2026.07 《Harness Engineering for Self-Improvement》· LangChain: 换 Harness, **52.8% → 66.5%**

Path 1 · RSI (recursive self-improvement) model side	Path 2 · Harness Engineering (self-modifying code) code side
Definition: AI systems iterating on themselves (including weights), each generation stronger Key concepts: - I.J. Good (1965) "Intelligence Explosion" — super-linear intelligence gain - Seed AI — a starting point able to launch the RSI loop - Takeoff — soft / medium / hard speed hypotheses Risks: - Alignment — goals drift from human intent - Control — safety must stay outside the loop	Definition: keep the weights; let the coding agent improve the Harness (code/MCP/Skill) Key data: - DGM evolutionary search: SWE-bench 20% → 50% (beyond hand-designed) - LangChain Terminal Bench: 52.8% → 66.5% - not one weight changed — pure Harness optimization Pros: - Controllable: every step is a reviewable, revertible diff - Safe: permissions/red lines stay outside the loop Cui Tianyi (DeepSeek): both paths are equally promising

Lilian Weng · three-level self-improvement path (deepening)



Key ideas: compensation migration (Weng) + the RSI red line

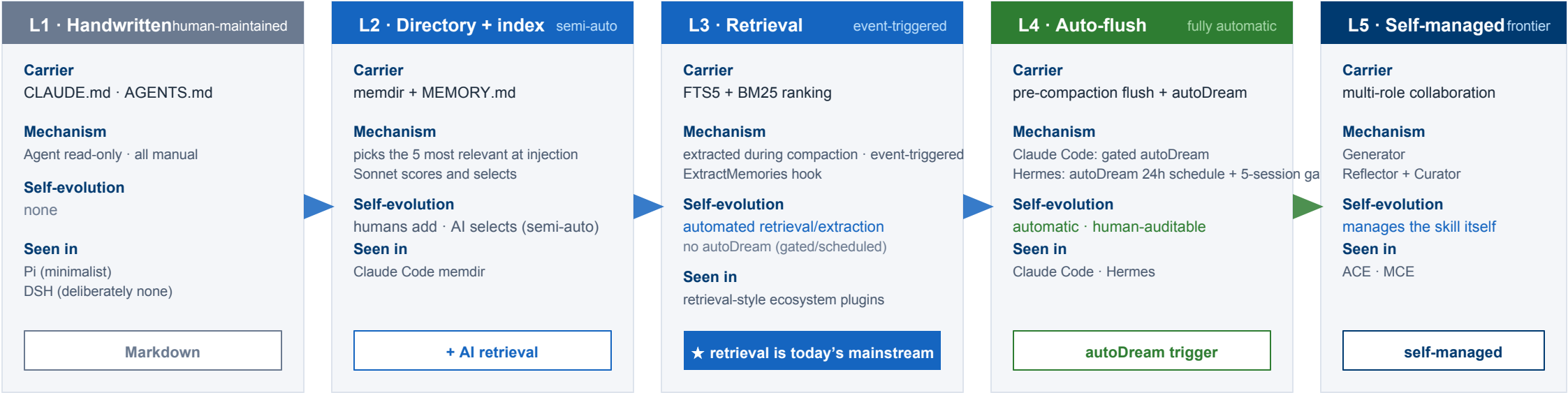
- A component exists because the model cannot do something · stronger models shift the weight
- RSI red line: permissions/security stay outside the loop (fail-closed discipline)

What matters is not Harness thickness but keeping up with the model



Memory Evolution: Five Rungs from Handwritten to Self-Managed

Six projects compared (details: Harness Dev Book §7.8) · automation deepens top-down: L1 manual → L5 self-managed (higher ≠ stronger; it is only automation)



Key insight: self-evolution is a capability gradient L1→L5, not just "growing"

- L1→L2: human-maintained → human-added (quantitative) · L2→L3: AI selects → AI writes (qualitative, event-triggered)
- L3→L4: inline extraction → pre-compaction flush (automated) · L4→L5: scheduled persistence → self-managed skills (frontier)

L3 retrieval aligns with Harness self-evolution (path 2) — both are L1→L5 progressions

Six-project self-evolution contrast

Harness-side vs model-side progressions · higher is not always better — DSH/Pi deliberately stay at L1





Two More Routes: DSH Restructurist · RL Parametrization

Two routes beyond growing · three trade-offs: grow / restructure / parameterize — the first two are safer until evaluators mature (Dev Book §7)

DSH: restructurist self-evolution	RL: parametrized self-evolution
Swap parts, don't grow them	written into the weights
Core idea: 押注composable structure + reconstructable logs	Core idea: 用 RL 把validated behaviors become weights
Three mechanisms ① The Agent Loop is a plugin · replaceable as a whole no code edits — swap the loop module ② Event sourcing · any session can fork / replay event sourcing = sessions fork and redo ③ Improvement by swapping parts, not growing each component upgrades independently	Two mainstream algorithms GRPO · GRPO: relative ranking in 8–16-sample groups no reward model needed (reward-model free) GiGPO · GiGPO: anchored states + step-level advantages tackles sparse rewards on 50+ step tasks Data source · no user-dialog data the real goal is distillation
Deliberately none: memory / skill self-evolution composability comes first	Limit: verifier maturity decides success be cautious before evaluators mature

Three trade-offs: grow · restructure · parameterize		
Grow grow memory / skills Seen in: Hermes · ACE/MCE Fits: exploration phase	Restructure swap parts instead of growing Seen in: DSH Fits: stable phase (composability bet)	Parameterize written into the weights Seen in: GRPO · GiGPO Fits: once verifiers mature

Six-project positions (today)

Hermes	★ grow + RL	DSH	Restructure	Codex	engineering pipeline	Claude Code	semi-auto	grow-style framework	ACE/MCE route	Pi	minimalist
--------	-------------	-----	-------------	-------	----------------------	-------------	-----------	----------------------	---------------	----	------------

Outlook: Where Harnesses Go Next

Frontier directions

Loop Engineering (Outer Loop orchestration)

Graph Engineering: Loops → Graphs (DAG control)

Nodes do work · edges route · control flow made explicit

One observation: compensation migration

A component exists = the model can't do it yet

Reference: Hermes dual self-evolution path (grow + RL)



Stronger models shift the compensation weight (Anthropic already dismantles early components)

Direction: better, maintainable constraints · Thank you — Q&A